Advanced Programming

Árpád Tamási · ELTE Faculty of Social Sciences

SESSION 6 — 13 OCTOBER 2026

SESSION 6

From today: 60 + 30

From now on, every session: **60 minutes active, 30 minutes consultation.**

Today: Testing · Code review · Homework 6

Testing

ChatGPT Ads Industry Dashboard

What it does. It groups ChatGPT ad accounts by industry and compares their spending, clicks and sales between industries.

What you found in Homework 4. The gap between industries looked bigger than the one real campaign's week-to-week swing: 143% versus 56%. But only one industry has real data; the others are example figures.

So right now: the industry differences on the chart are not yet real.

A test would go red if …

MEASURED

Central European air traffic

What it does. It records planes over Central Europe on a schedule, so traffic over time can be asked about later.

What you found in Homework 4. You counted the fully captured hours over two weeks: only 23 of 337.

So right now: Answers about traffic over time would rest on data that is mostly gaps.

A test would go red if …

MEASURED

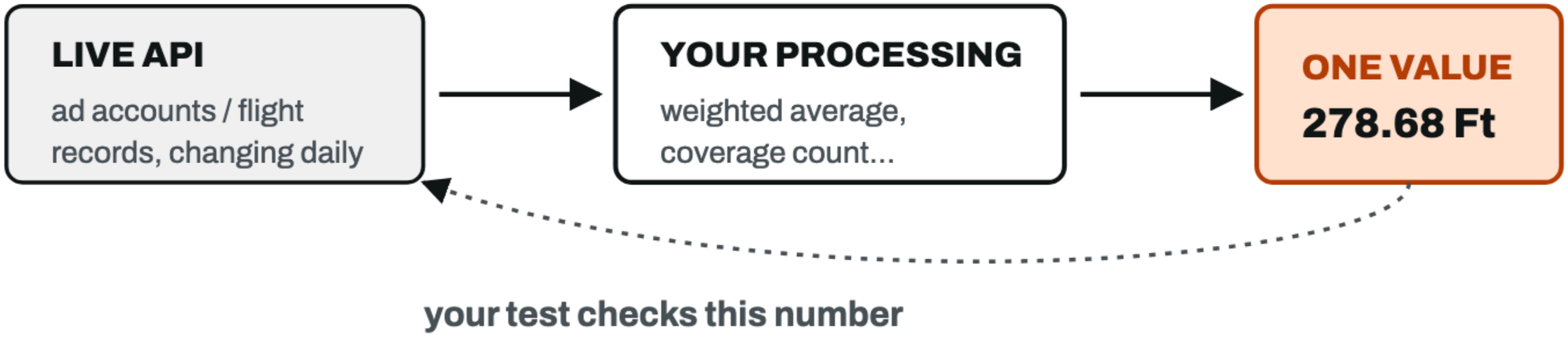
Homework 5, for both

	THE QUESTION
Ads Dashboard	Two real accounts, 8–21 September: what is the weighted CTR and CPC?
Air traffic	How many of 337 hours were fully captured?

Both questions have the same shape: **pull from a live API, process it, get a number.**

MEASURED

What happens



What's wrong with that

```
assert weighted_cpc() == 278.68      # copied from running the code
```

We want to test the processing. But the processing's output depends on the API's output — and the API's output changes. Test the whole chain, and you're not testing your logic. You're testing today's weather.

The fix: unit test with constants

```php
public function test_weighted_average_combines_two_accounts() {
    // numbers YOU invent — not from the API, not from the code
    $accounts = [
        ['impressions' => 100, 'clicks' => 10, 'spend' => 500],
        ['impressions' => 200, 'clicks' => 30, 'spend' => 900],
    ];
    // by hand: (10 + 30) / (100 + 200) = 0.1333
    $this->assertEqualsWithDelta(0.1333, weighted_ctr($accounts), 0.0001);
}
```

Skip the API. Invent the numbers, work out the answer by hand.

Two kinds of test

	UNIT TEST	INTEGRATION TEST
Input	numbers you invent, frozen	the real API, the real database
Checks	your logic does the math right	the pieces actually connect
Changes with the world?	never	yes — that's the point

Both matter. Neither replaces the other. The unit test can't tell you the API still works. The integration test can't tell you your math is right — it can only tell you something came back.

CLAIM

Why we test

A test says what the code should do, decided before you look at what it does.

CLAIM

Why we test

> **“If you have tests, you do not fear making changes to the code! Without tests every change is a possible bug.”**

Robert C. Martin, Clean Code, 2008, chapter 9.

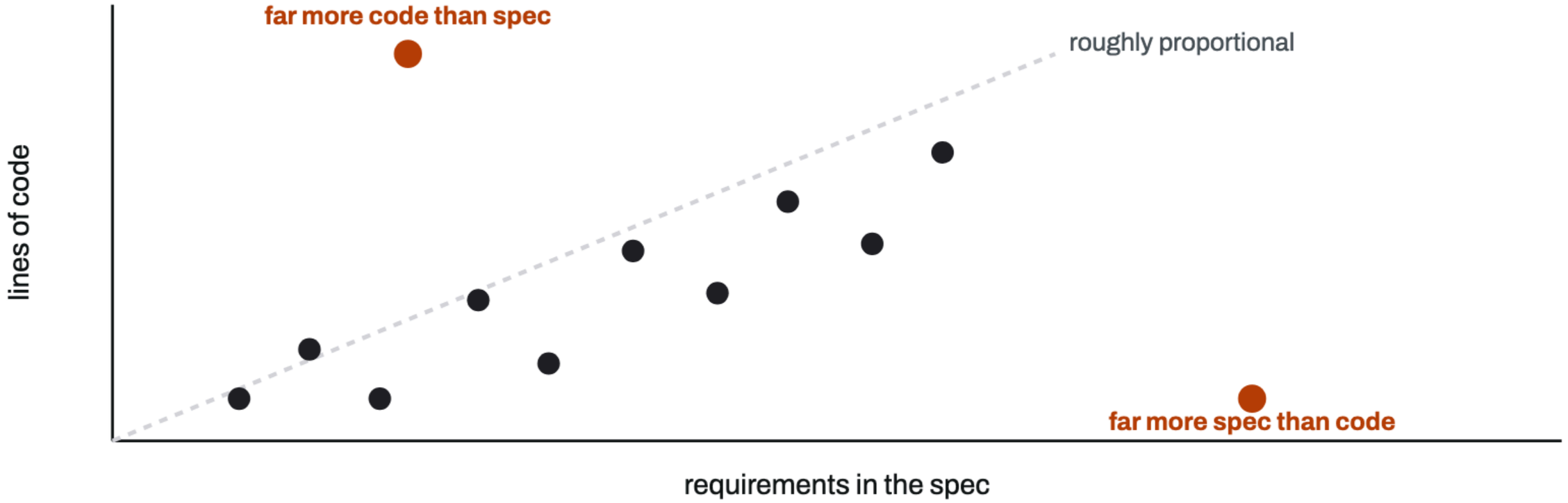
Code review

2,200 lines

The median project in this course. The agent wrote most of it, faster than anyone can read it, and nobody has read it as a whole.

ILLUSTRATION

Has the code drifted from the spec?



Most projects sit near the line. The two marked points didn’t: one has code the spec never described, the other has spec the code never built.

ILLUSTRATION

Names that aren't in the spec

```python
# route_planner.py
def apply_slack_buffer(route, confidence_band=0.15):
    ...
```

Where did `slack_buffer` come from? Not the spec, not a conversation you remember having. The agent decided your routes needed one, built it, and now other code depends on it.

ILLUSTRATION

Patched where it broke, not where it's wrong

```
# commit 1: "assume every account reports daily"
# commit 2: "fix: handle accounts that report weekly"
# commit 3: "fix: handle accounts that report monthly too"
# commit 4: "fix: one more edge case for irregular reporting"
```

The assumption was wrong from commit 1. Each time it broke, the agent patched the symptom it hit — never went back and fixed the rule underneath. Four commits later, nobody could state the actual rule.

A copy instead of a change

```
store.py
app/store.py
```

Two copies of the same module, now 148 lines apart. The agent never sees the whole codebase at once. Asked for something similar, it builds it again instead of finding what's there.

The fix is yours to see: look for units — modules, components, functions — that do almost the same job under different names. Then keep one.

Errors that disappear

```
def load_rates(path):
    try:
        return read_csv(path)
    except Exception:
        return None      # the program carries on, with nothing
```

The program looks as if it works. The error shows up three functions later as an empty chart, or never. The agent writes this because it was asked to make the code run.

Look for: `except Exception`, `except: pass`, and `return None` after an error.

Things nobody chose

DEPENDENCIES

- Does the package **exist**? Models invent plausible names, and attackers register them.
- Is it **used**? Is it **pinned** to one exact version?

DEAD CODE

- A function the agent wrote and nothing calls.
- The same feature, built twice under different names.

Spracklen et al., We Have a Package for You!, USENIX Security 2025: across 16 code models and 576,000 samples, about one in five recommended packages did not exist.

ILLUSTRATION

The quiet default

```
timeout = config.get("timeout", 0)    # 0 means "no timeout" downstream
```

A missing value became a decision, silently. If `timeout` was never meant to be optional, this should be an error, not a fallback nobody chose.

Look for: `.get(key, default)` where a missing key is actually a bug; defaults that change behavior instead of signaling a problem.

What should not be in git

- `data/raw/…json` — large raw data files, committed
- keys, `.env`, real people's data

A plain `git rm` doesn't remove it. The file stays in every earlier commit — anyone can still check it out. Getting it out for real means rewriting the whole history with a tool such as `git filter-repo`, which breaks everyone else's copy.

Homework 6

Homework 6 — review your whole codebase

1. Start a new branch.
2. Run specialised reviews on your whole repository, one at a time — today's slides are your starting list.
3. Each review, and what you did about it, goes in its own commit.
4. Merge the branch back. Don't delete it.

Due **Saturday 17 October, 20:00**. Late until Monday 19 October, 20:00, for half credit.